Hot, fresh, and made with love

Pizza Mania

Sales Analysis

linkedin.com/in/vishal-mahavar-0a49043b3
github.com/vishal-tech3

By Vishal

Table of content:-

1. Business Problem ---------- 4

2. Dataset Information---------- 5

3. Data Cleaning & null handling----------7

4. Basic Analysis with KPI's ---------- 10

5. Exploratory data analysis---------- 15

6. Advance analysis---------- 23

7. Dashboard & Visualizations---------- 29

7.1 Data insights

7.2 Recommendations

8. Skills & tools used ----------38

9. Outcomes ----------39

10. Author details ----------40

Project Workflow:

1. Business problem
    - Understanding business context
    - problems
2. Understanding Data
    - Schema
    - Tables
    - Relations
3. Data cleaning:
    - Fixed data types
    - Handled null values
    - Ensured relational and referential integrity
4. Data Analysis
    - KPI calculation
    - Trend analysis
    - Customer behavior analysis
    - EDA
    - Product performance
5. Dashboard creation (Excel)
    - KPI cards
    - Revenue trends
    - Peak hour analysis
    - Category, size & type wise breakdown

Business Problem:

The objective of this project is to identify key business problems such as The company may lack visibility into **revenue trends**, **underperforming menu items** that reduce profitability and lead to **insufficient menu management** , There may be limited understanding of **customer demand patterns** including peak ordering hours and days, resulting in **missed sales opportunities**, additionally unclear insight into **customer ordering behavior** can hinder effective pricing promotional strategies, The business might also struggle with **pricing inefficiencies** where higher price items do not generate expected sales and **category imbalances** , low selling items contribute to **inventory waste**, **data related issues**, such as **multiple entries** or **redundancies** can create **inaccurate KPI** calculations and also effects **decision making**.

Business Context

- The business lacks clarity on revenue drivers
- Customer demand patterns
- Product performance
- Leading to missed growth opportunities.

Objective

- The objective is to identify what drives revenue and where the business is losing potential sales.

#.Dataset Information :

Pizza mania is a database created in mysql in order to solve real world problem related to many fast-food industries which serve pizza as a primary source of revenue and also the industries which serve beverages and fast food.

#1. Schema : **pizza_mania**

Rows : 48620 Columns : 15 Tables : 4

#2. **Tables** used in this project:

- **Order_details**

  order_details_id, : unique order id for every row
  order_id, : includes multiple orders on same id
  pizza_id, : combination of pizza_id and pizza_type_id
  quantity : quantity of every order

- **Orders**

  order_id, : includes multiple orders on same id
  order_date, : date of every confirmed order
  order_time : time of every confirmed order

- **Pizzas**

  pizza_id, : combination of pizza_id and pizza_type_id
  pizza_type_id, : combination of pizza name and category
  size, : sizes are S, M, L, XL, XXL
  price : price of every quantity ordered

- **Pizza_types**

  pizza_type_id, : combination of pizza name and category
  name, : name of pizza
  category, : category Chicken, Classic, Veggie, Supreme
  ingredients : ingredients used in making of pizza

Tables:

Order_details

order_details_id	order_id	pizza_id	quantity
1	1	hawaiian_m	1
2	2	classic_dlx_m	1
3	2	five_cheese_l	1
4	2	ital_supr_l	1
5	2	mexicana_m	1
6	2	thai_ckn_l	1
7	3	ital_supr_m	1
8	3	prsc_argla_l	1
9	4	ital_supr_m	1
10	5	ital_supr_m	1
11	6	bbq_ckn_s	1
12	6	the_greek_s	1
13	7	spinach_sup...	1
14	8	spinach_sup...	1
15	9	classic_dlx_s	1
16	9	green_garde...	1

Orders

order_id	order_date	order_time
1	2015-01-01	11:38:36
2	2015-01-01	11:57:40
3	2015-01-01	12:12:28
4	2015-01-01	12:16:31
5	2015-01-01	12:21:30
6	2015-01-01	12:29:36
7	2015-01-01	12:50:37
8	2015-01-01	12:51:37
9	2015-01-01	12:52:01
10	2015-01-01	13:00:15
11	2015-01-01	13:02:59
12	2015-01-01	13:04:41
13	2015-01-01	13:11:55
14	2015-01-01	13:14:19
15	2015-01-01	13:33:00
16	2015-01-01	13:34:07

Pizzas

pizza_id	pizza_type_id	size	price
bbq_ckn_s	bbq_ckn	S	12.75
bbq_ckn_m	bbq_ckn	M	16.75
bbq_ckn_l	bbq_ckn	L	20.75
cali_ckn_s	cali_ckn	S	12.75
cali_ckn_m	cali_ckn	M	16.75
cali_ckn_l	cali_ckn	L	20.75
ckn_alfredo_s	ckn_alfredo	S	12.75
ckn_alfredo_m	ckn_alfredo	M	16.75
ckn_alfredo_l	ckn_alfredo	L	20.75
ckn_pesto_s	ckn_pesto	S	12.75
ckn_pesto_m	ckn_pesto	M	16.75
ckn_pesto_l	ckn_pesto	L	20.75
southw_ckn_s	southw_ckn	S	12.75
southw_ckn_m	southw_ckn	M	16.75
southw_ckn_l	southw_ckn	L	20.75
thai_ckn_s	thai_ckn	S	12.75

Pizza_types

pizza_type_id	name	category	ingredients
bbq_ckn	The Barbecue Chicken Pizza	Chicken	Barbecued Chicken, Red Pe...
cali_ckn	The California Chicken Pizza	Chicken	Chicken, Artichoke, Spinach,...
ckn_alfredo	The Chicken Alfredo Pizza	Chicken	Chicken, Red Onions, Red P...
ckn_pesto	The Chicken Pesto Pizza	Chicken	Chicken, Tomatoes, Red Pe...
southw_ckn	The Southwest Chicken Pizza	Chicken	Chicken, Tomatoes, Red Pe...
thai_ckn	The Thai Chicken Pizza	Chicken	Chicken, Pineapple, Tomato...
big_meat	The Big Meat Pizza	Classic	Bacon, Pepperoni, Italian Sa...
classic_dlx	The Classic Deluxe Pizza	Classic	Pepperoni, Mushrooms, Red...
hawaiian	The Hawaiian Pizza	Classic	Sliced Ham, Pineapple, Mozz...
ital_cpcllo	The Italian Capocollo Pizza	Classic	Capocollo, Red Peppers, To...
napolitana	The Napolitana Pizza	Classic	Tomatoes, Anchovies, Gree...
pep_msh_pep	The Pepperoni, Mushroom, ...	Classic	Pepperoni, Mushrooms, Gre...
pepperoni	The Pepperoni Pizza	Classic	Mozzarella Cheese, Pepperoni
the_greek	The Greek Pizza	Classic	Kalamata Olives, Feta Chees...
brie_carre	The Brie Carre Pizza	Supreme	Brie Carre Cheese, Prosciutt...
calabrese	The Calabrese Pizza	Supreme	'Nduja Salami, Pancetta, To...

#. Data Cleaning, null handling & ensured relational integrity:

```
# Data type correction
SET SQL_SAFE_UPDATES = 0;

ALTER TABLE pizza_types
MODIFY pizza_type_id VARCHAR(50);

ALTER TABLE pizzas
MODIFY pizza_type_id VARCHAR(50);


# Primary key
alter table pizza_types
add primary key (pizza_type_id);

# Foreign key
alter table pizzas
add constraint fk_pizza_type
foreign key (pizza_type_id)
references pizza_types(pizza_type_id);

# Standardize date and time with safe updates off


UPDATE orders
SET order_time = STR_TO_DATE(order_time, '%H:%i:%s');

UPDATE orders
SET order_date = STR_TO_DATE(order_date, '%y-%m-%d');

UPDATE orders
SET order_time = '00:00:00'
WHERE order_time IS NULL;


SET SQL_SAFE_UPDATES = 1;
```

Null handling & join validation :

```
# Null checking for every table
SELECT
    COUNT(*) AS total_rows,
    SUM(CASE WHEN order_details_id IS NULL THEN 1 ELSE 0 END) AS
order_id_nulls,
    SUM(CASE WHEN order_id IS NULL THEN 1 ELSE 0 END) AS order_id_nulls,
    SUM(CASE WHEN pizza_id IS NULL THEN 1 ELSE 0 END) AS pizza_id_nulls,
    SUM(CASE WHEN quantity IS NULL THEN 1 ELSE 0 END) AS quantity_nulls
FROM order_details;

SELECT
    count(*) as total_rows,
    sum(CASE WHEN order_id IS NULL THEN 1 ELSE 0 END) AS order_id_nulls,
    sum(CASE WHEN order_id IS NULL THEN 1 ELSE 0 END) AS order_date_nulls,
    sum(CASE WHEN order_id IS NULL THEN 1 ELSE 0 END) AS order_time_nulls
FROM orders;

SELECT
    count(*) as total_rows,
    sum(CASE WHEN pizza_id IS NULL THEN 1 ELSE 0 END) AS pizza_id_nulls,
    sum(CASE WHEN pizza_type_id IS NULL THEN 1 ELSE 0 END) AS
pizza_type_id_nulls,
    sum(CASE WHEN size IS NULL THEN 1 ELSE 0 END) AS size_nulls,
    sum(CASE WHEN price IS NULL THEN 1 ELSE 0 END) AS price_nulls
FROM pizzas;

SELECT
    count(*) as total_rows,
    sum(CASE WHEN pizza_type_id IS NULL THEN 1 ELSE 0 END) AS
pizza_type_id_nulls,
    sum(CASE WHEN name IS NULL THEN 1 ELSE 0 END) AS pizza_type_id_nulls,
    sum(CASE WHEN category IS NULL THEN 1 ELSE 0 END) AS pizza_type_id_nulls,
    sum(CASE WHEN ingredients IS NULL THEN 1 ELSE 0 END) AS
pizza_type_id_nulls
FROM pizza_types;
```

```sql
# Updating handleable null values

SET SQL_SAFE_UPDATES = 0;

UPDATE pizza_types
SET   category = 'others'
WHERE
     category IS NULL;

UPDATE pizza_types
SET    name = 'unknown'
WHERE
     name IS NULL;

UPDATE pizza_types
SET    ingredients = 'not available'
WHERE

     ingredients IS NULL;

# Deleting non-handleable null values


DELETE FROM orders
WHERE
     order_time IS NULL OR
     order_date IS NULL OR
     order_id IS NULL;

DELETE FROM order_details
WHERE
    order_details_id IS NULL
    OR order_id IS NULL
    OR pizza_id IS NULL
    OR quantity IS NULL;

SET SQL_SAFE_UPDATES = 1;
```

#.Basic Analysis with KPI's:

SQL Queries

```
use pizza_mania;
SELECT * FROM  orders;
```

#1. Retrieve the total number of orders placed.

```
SELECT
    COUNT(*) AS total_orders
FROM
    orders;
```

total_orders
21350

There are total 21350 different orders

#2. Calculate the total revenue generated from pizza sales.

```
SELECT
    SUM(od.quantity * p.price) AS Total_sales
FROM
    order_details od
        JOIN
    pizzas p ON od.pizza_id = p.pizza_id;
```

Total_sales
817860.049999993

Total revenue generate as on 31-Dec-2015 is **$817860.05**

#.3 Identify the highest-priced pizza.

```
SELECT
    name, price
FROM
    pizza_types pt
        JOIN
    pizzas p ON pt.pizza_type_id = p.pizza_type_id
ORDER BY price DESC
LIMIT 1;
```

name	price
The Greek Pizza	35.95

"The Greek Pizza" is the highest price pizza

#.4 Identify the most common pizza size ordered.

```
SELECT
    size, COUNT(DISTINCT order_id) AS total_orders
FROM
    order_details od
        JOIN
    pizzas p ON od.pizza_id = p.pizza_id
GROUP BY size
ORDER BY total_orders DESC
LIMIT 1;
```

size	total_orders
L	12736

Large pizzas are mostly ordered.

#5. KPI's of orders placed.

```
SELECT
    COUNT(DISTINCT od.order_id) total_orders,
    SUM(price * quantity) AS total_revenue,
    ROUND(SUM(price * quantity) / COUNT(DISTINCT
od.order_id),
            2) AS avg_order_value,
    SUM(quantity) AS total_pizzas_sold
FROM
    orders o
        JOIN
    order_details od ON o.order_id = od.order_id
        JOIN
    pizzas p ON od.pizza_id = p.pizza_id;
```

total_orders	total_revenue	avg_order_value	total_pizzas_sold
21350	817860.049999993	38.31	49574

There are total **21350 order placed**, **49574 pizzas sold** and over **817860 revenue generated** over year 2015 with **average order value $38.31**

#6. Revenue per Pizza

```
select
    round(sum(price*quantity)/ sum(quantity),2) as
revenue_per_pizza
from order_details od join pizzas p on
od.pizza_id=p.pizza_id;
```

revenue_per_pizza
16.5

Average revenue generated over pizza quantity is **$16.5**

#.7 List the top 5 most ordered pizza types along with their quantities.

```
SELECT
    name, SUM(quantity) AS total_qty_ordered
FROM
    order_details od
        JOIN
    pizzas p ON od.pizza_id = p.pizza_id
        JOIN
    pizza_types pt ON p.pizza_type_id =
pt.pizza_type_id
GROUP BY name
ORDER BY total_qty_ordered DESC
LIMIT 5;
```

name	total_qty_ordered
The Classic Deluxe Pizza	2453
The Barbecue Chicken Pizza	2432
The Hawaiian Pizza	2422
The Pepperoni Pizza	2418
The Thai Chicken Pizza	2371

Most ordered pizza quantity is from classic deluxe.

#.8 List the worst 5 ordered pizza types along with their quantities.

```
SELECT
    pt.name,
    SUM(od.quantity) AS total_qty
FROM order_details od
JOIN pizzas p ON od.pizza_id = p.pizza_id
JOIN pizza_types pt ON p.pizza_type_id =
pt.pizza_type_id
GROUP BY pt.name
ORDER BY total_qty ASC LIMIT 5;
```

name	total_qty
The Brie Carre Pizza	490
The Mediterranean Pizza	934
The Calabrese Pizza	937
The Spinach Supreme Pizza	950
The Soppressata Pizza	961

Here are the 5 least ordered pizzas

#.Exploratory Data Analysis:

#.9 Analyze small set of pizzas (~20%) generating (~80%) of the revenue.

```
with pizza_revenue as (
    select
        pt.name as pizza_name,
        sum(price*quantity) as revenue
    from order_details od join pizzas p on
od.pizza_id=p.pizza_id join pizza_types pt on p.pizza_type_id=
pt.pizza_type_id
    group by pt.name
),

ranked as (
    select
        pizza_name,
        revenue,
        sum(revenue) over (order by revenue desc) as
cumulative_revenue,
        sum(revenue) over () as total_revenue
    from pizza_revenue
)

select
    pizza_name,
    revenue,
    round(cumulative_revenue*100/total_revenue,2) as
cumulative_pct
from ranked
order by revenue desc;
```

pizza_name	revenue	cumulative_pct	row_num
The Thai Chicken Pizza	43434.25	5.31	1
The Barbecue Chicken Pizza	42768	10.54	2
The California Chicken Pizza	41409.5	15.6	3
The Classic Deluxe Pizza	38180.5	20.27	4
The Spicy Italian Pizza	34831.25	24.53	5
The Southwest Chicken Pizza	34705.75	28.77	6
The Italian Supreme Pizza	33476.75	32.87	7
The Hawaiian Pizza	32273.25	36.81	8
The Four Cheese Pizza	32265.70000000065	40.76	9
The Sicilian Pizza	30940.5	44.54	10
The Pepperoni Pizza	30161.75	48.23	11
The Greek Pizza	28454.1000000000...	51.71	12
The Mexicana Pizza	26780.75	54.98	13
The Five Cheese Pizza	26066.5	58.17	14
The Pepper Salami Pizza	25529	61.29	15
The Italian Capocollo Pizza	25094	64.36	16
The Vegetables + Vegetable...	24374.75	67.34	17
The Prosciutto and Arugula ...	24193.25	70.3	18
The Napolitana Pizza	24087	73.24	19
The Spinach and Feta Pizza	23271.25	76.09	20
The Big Meat Pizza	22968	78.9	21

Here are the top (~65%) pizzas generating (~80%) of the total revenue.

#.10 Join relevant tables to find the category-wise distribution of pizzas.

```
SELECT
    category,
    COUNT(name) AS total
FROM pizza_types GROUP BY category;
```

category	total
Chicken	6
Classic	8
Supreme	9
Veggie	9

Here are total 27 pizzas over 4 categories

#.11 Join the necessary tables to find the total quantity of each pizza category ordered.

```
SELECT
    category, SUM(quantity) AS total_qty_ordered
FROM
    order_details od
        JOIN
    pizzas p ON od.pizza_id = p.pizza_id
        JOIN
    pizza_types pt ON p.pizza_type_id =
pt.pizza_type_id
GROUP BY category;
```

category	total_qty_ordered
Classic	14888
Veggie	11649
Supreme	11987
Chicken	11050

Classic category is the strongest asset, it has the highest brand recognition and customer trust, Veggie and supreme suggest a balance demand, Chicken is trailing

#12. Peak Day Analysis

```
SELECT
    DAYNAME(order_date) AS day_name,
    COUNT(DISTINCT order_id) AS total_orders,
    concat(round(COUNT(order_id)*100/(select
COUNT(DISTINCT order_id) from orders),2),"%") as PCT
FROM orders
GROUP BY day_name
ORDER BY total_orders DESC;
```

day_name	total_orders	PCT
Friday	3538	16.57%
Thursday	3239	15.17%
Saturday	3158	14.79%
Wednesday	3024	14.16%
Tuesday	2973	13.93%
Monday	2794	13.09%
Sunday	2624	12.29%

On (Friday, Thursday, Saturday, Wednesday) these day order distribution is 60.69% and 39.31% is for the rest days.

#13. Revenue Growth % (Day-over-Day)

```
select
    order_date,
    round(daily_revenue,2) as daily_revenue,
    round((daily_revenue - lag(daily_revenue) over
(order by order_date))
    / lag(daily_revenue) over(order by
order_date)*100,2) as growth_pct
from
(select order_date,
        sum(quantity*price) as daily_revenue
from orders o join order_details od on
o.order_id=od.order_id
        join pizzas p on od.pizza_id=p.pizza_id
        group by o.order_date)  daily_rev;
```

order_date	daily_revenue	growth_pct
2015-01-01	2713.85	NULL
2015-01-02	2731.9	0.67
2015-01-03	2662.4	-2.54
2015-01-04	1755.45	-34.07
2015-01-05	2065.95	17.69
2015-01-06	2428.95	17.57
2015-01-07	2202.2	-9.34
2015-01-08	2838.35	28.89
2015-01-09	2127.35	-25.05
2015-01-10	2463.95	15.82
2015-01-11	1872.3	-24.01
2015-01-12	1919.05	2.5
2015-01-13	2049.6	6.8
2015-01-14	2527.4	23.31
2015-01-15	1984.8	-21.47

It shows Day-by-day percentage revenue growth

#.14 Determine the distribution of orders by hour of the day.

```
SELECT
    HOUR(order_time) AS Time, COUNT(DISTINCT order_id)
AS Total_orders
FROM
    orders
GROUP BY HOUR(order_time);
```

Time	total_orders
9	1
10	8
11	1231
12	2520
13	2455
14	1472
15	1468
16	1920
17	2336
18	2399
19	2009
20	1642
21	1198
22	663
23	28

Most orders are seen from 11am to 9pm

#.15 Group the orders by date and calculate the average number of pizzas ordered per day.

```
SELECT

    ROUND(AVG(qty),2) AS avg_order_value

FROM

    (SELECT

        o.order_date AS date, SUM(od.quantity) AS qty

    FROM

        order_details od

    JOIN orders o ON od.order_id = o.order_id

    GROUP BY o.order_date) AS order_qty;
```

avg_order_value
138.47

Here is average 138 quantity of pizza ordered pe day

#.16 Determine the top 3 most ordered pizza types based on revenue.

```
SELECT
    pt.name AS Pizza_name,
    COUNT(DISTINCT order_id) AS Total_orders,
    SUM(quantity) AS Total_quantity_ordered,
    CAST(SUM(quantity * price) AS DECIMAL (10 , 2 ))
AS revenue
FROM
    order_details od
        JOIN
    pizzas p ON od.pizza_id = p.pizza_id
        JOIN
    pizza_types pt ON p.pizza_type_id =
pt.pizza_type_id
GROUP BY pt.name
ORDER BY revenue DESC
LIMIT 3;
```

Pizza_name	Total_orders	Total_quantity_ordered	revenue
The Thai Chicken Pizza	2315	2371	43434.25
The Barbecue Chicken Pizza	2372	2432	42768.00
The California Chicken Pizza	2302	2370	41409.50

most revenue of orders are form these 3 pizza categories

#. Advanced Analysis:

#.17 Calculate the percentage contribution of each pizza type to total revenue.

```
with cte (name,revenue) as(
select
     pt.category as name,
     cast(sum(quantity*price) as decimal (10,2)) as
revenue
     from order_details od
          join pizzas p on od.pizza_id=p.pizza_id
          join pizza_types  pt
               on p.pizza_type_id= pt.pizza_type_id
group by pt.category
)

select
     name,
     revenue*100/(select cast(sum(quantity*price) as
decimal (10,2)) as revenue
from
     order_details od join pizzas p
          on od.pizza_id=p.pizza_id
          join pizza_types  pt
          on p.pizza_type_id= pt.pizza_type_id) as PCT
from cte
order by revenue desc;
```

name	PCT
Classic	26.905960
Supreme	25.456311
Chicken	23.955138
Veggie	23.682591

Classic share most of the revenue distribution while veggie is the least of it.

#.18 Contribution % of Each Pizza

```
SELECT
    pt.name,
    ROUND(SUM(od.quantity * p.price) * 100 /
        (SELECT SUM(od2.quantity * p2.price)
         FROM order_details od2
         JOIN pizzas p2 ON od2.pizza_id =
p2.pizza_id), 2) AS contribution_pct
FROM order_details od
JOIN pizzas p ON od.pizza_id = p.pizza_id
JOIN pizza_types pt ON p.pizza_type_id =
pt.pizza_type_id
GROUP BY pt.name
ORDER BY contribution_pct DESC;
```

name	contribution_pct
The Thai Chicken Pizza	5.31
The Barbecue Chicken Pizza	5.23
The California Chicken Pizza	5.06
The Classic Deluxe Pizza	4.67
The Spicy Italian Pizza	4.26
The Southwest Chicken Pizza	4.24
The Italian Supreme Pizza	4.09
The Hawaiian Pizza	3.95
The Four Cheese Pizza	3.95
The Sicilian Pizza	3.78
The Pepperoni Pizza	3.69
The Greek Pizza	3.48
The Mexicana Pizza	3.27
The Five Cheese Pizza	3.19

Here is percentage wise distribution of pizzas over revenue

#.19 Size-wise Revenue Share

```
with rev as (
select
    size,
    sum(price*quantity) as revenue
from order_details od join pizzas p on
od.pizza_id=p.pizza_id group by size
)

select
    size,
    revenue*100/(select sum(price*quantity) from
order_details od join pizzas p on od.pizza_id=p.pizza_id)
as PCT   from rev;
```

size	pct
M	30.49
L	45.89
S	21.77
XL	1.72
XXL	0.12

Here is percentage wise distribution of revenue share of Large size is the most while XXL and XL are the least

#.20 Peak vs Non-Peak Revenue

```
with rev as (
select
    sum(price*quantity) as revenue
from order_details od join pizzas p on
od.pizza_id=p.pizza_id
)

select
    case
        when hour(order_time) between 12 and 15 then
"Lunch"
        when hour(order_time) between 18 and 22 then
"Dinner"
        else "Off-peak"
    end as Time_slot,
        sum(price*quantity) as revenue,
        round(sum(price*quantity)*100/(select revenue
from rev),2) as PCT
from orders o
JOIN order_details od ON o.order_id = od.order_id
JOIN pizzas p ON od.pizza_id = p.pizza_id
group by time_slot;
```

Time_slot	revenue	PCT
Off-peak	202736.65000000037	24.79
Lunch	330137.3000000035	40.37
Dinner	284986.10000000196	34.85

Revenue distributed over Peak vs Non-peak hours, during 'Lunch' peak hours revenue distribution is the most 40.37% and followed by Dinner peak hour with 34.85% then Off-peak hours which is the lowest 24.79%

#.21 Determine the top 3 most ordered pizza types based on revenue for each pizza category.

```
SELECT
     CATEGORY,
     NAME,
     revenue
FROM
     (SELECT
          category,
          NAME,
          total_orders,
          revenue,
         rank() OVER(PARTITION BY category ORDER BY revenue
     DESC) AS rnk
         FROM
     (SELECT
          pt.category,
          pt.name,
          COUNT(DISTINCT order_id) total_orders,
          CAST(SUM(quantity*price) AS DECIMAL (10,2)) AS
          revenue
     FROM
          order_details od JOIN pizzas p
               ON od.pizza_id=p.pizza_id
          JOIN pizza_types  pt
               ON p.pizza_type_id= pt.pizza_type_id
          GROUP BY pt.category,pt.name) AS cat_total) AS
     cat_rank

     WHERE rnk <=3;
```

CATEGORY	NAME	revenue
Chicken	The Thai Chicken Pizza	43434.25
Chicken	The Barbecue Chicken Pizza	42768.00
Chicken	The California Chicken Pizza	41409.50
Classic	The Classic Deluxe Pizza	38180.50
Classic	The Hawaiian Pizza	32273.25
Classic	The Pepperoni Pizza	30161.75
Supreme	The Spicy Italian Pizza	34831.25
Supreme	The Italian Supreme Pizza	33476.75
Supreme	The Sicilian Pizza	30940.50
Veggie	The Four Cheese Pizza	32265.70
Veggie	The Mexicana Pizza	26780.75
Veggie	The Five Cheese Pizza	26066.50

Here are the top 3 sales of each pizza category which shares approx 50% of the sales

#.Dashboard & Visualizations:

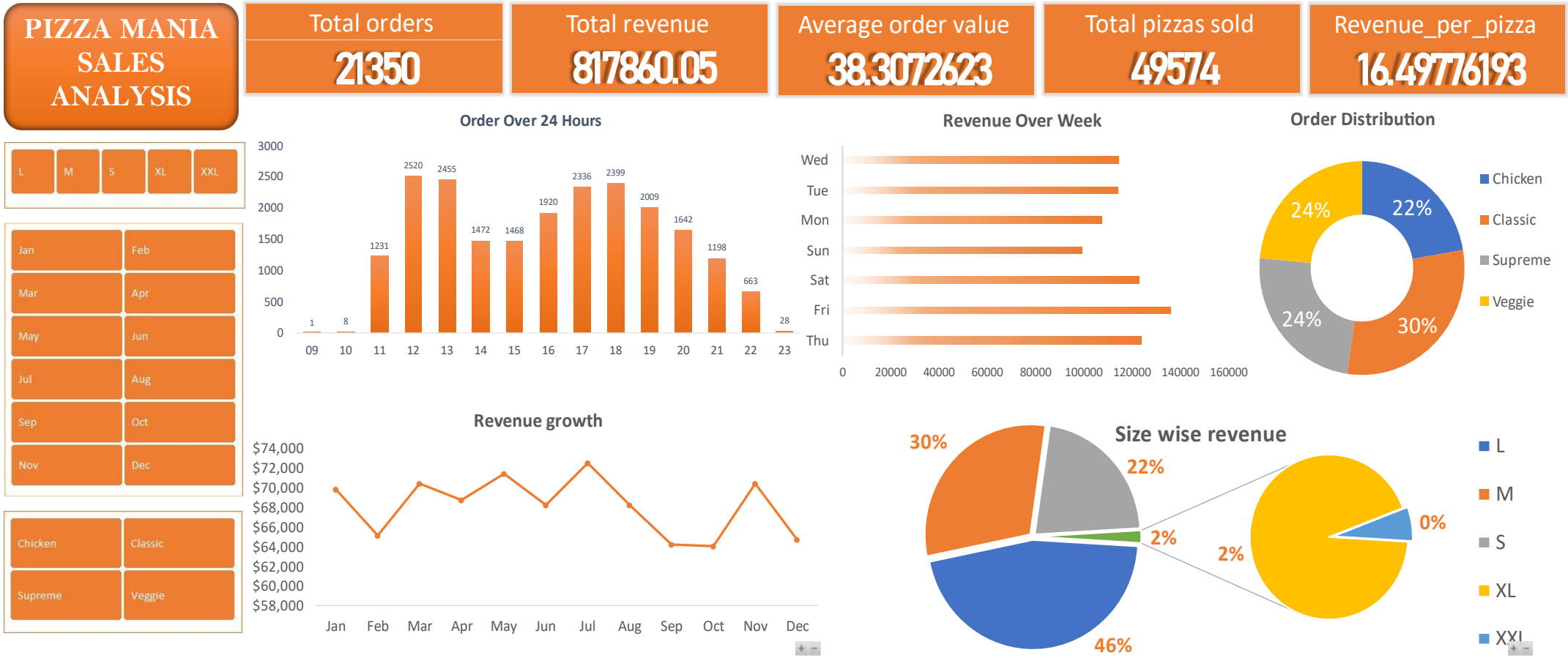
Key progression indications-

Total orders	**21350**
Total revenue	**$ 817860.05**
Average order value	**$ 38.3072623**
Total pizzas sold	**49574**
Revenue per pizza	**$16.49776193**

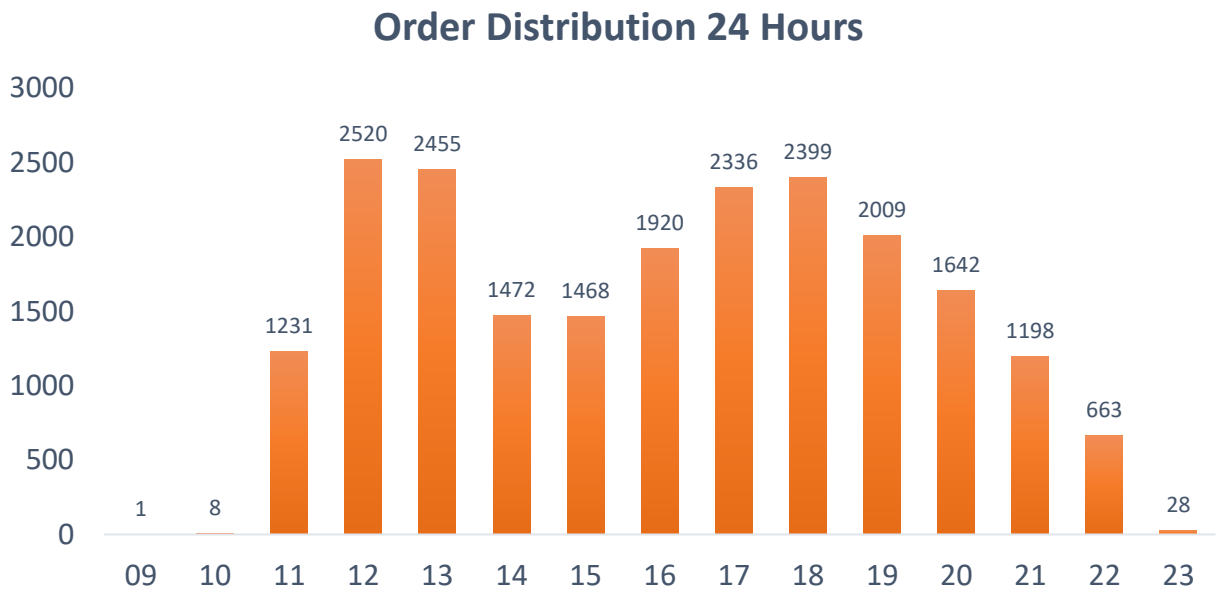
Key Progression Indications of 2015 sales summary:

There is **21350** different **orders** ordered in **2015** with **Total revenue generated $817860.05**, **Average order value** (total revenue*100/total different orders) was **$38.30**, Total **49574 quantity of pizzas sold**, **Average Revenue of each pizza** (total revenue*100/total quantity) was **$16.50.**

Dashboard:



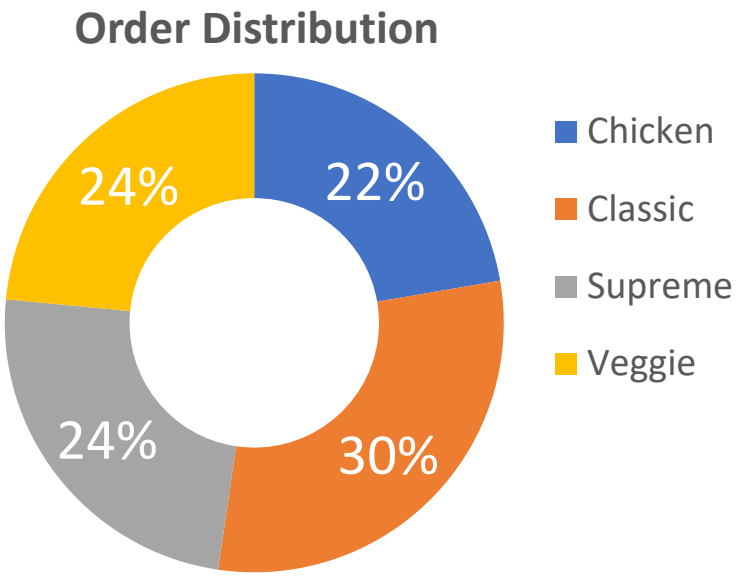
"Pizza Mania's revenue is primarily driven by peak-hour demand, large-size pizzas, and a small group of top-performing products, while off-peak hours and underperforming categories represent key growth opportunities."



- Key Insights:
    - Most orders are concentrated between 11am to 9pm with clear peak during
    - Lunch (12pm – 1pm)
    - Evening (4pm – 7pm)
    - Off peak hours shows a sharp decline in orders after 10 pm and before 11am and mid between 2 & 3pm
- Recommendation:
    - Ensure strongest and most experienced staff is on ground during peak hours.
    - Volume drop sharply after 10pm analyze the labor cost versus the revenue of the 28 orders placed at 11 to see if closing an hour earlier is a more profitable move
    - The lunch peak is a sharper and more condensed than evening ensure ingredients and raw materials are available at these hours so the kitchen ensure smooth operation when rush doubles at noon.

- **Key Insights:**
    - Revenue trend shows inconsistent demand over months.
    - Peak performance is observed in march may July and November .
    - Significant dips are observed in fab august - oct.

- **Recommendation:**
    - Low performing months
        - Introduce Seasonal promotions and discount.
        - launch happy hours and bundle deals in slow periods and limited time offer to create urgency, reduce waste during low months.
    - High performing months
        - focus on top selling items, remove and redesign for low performing items
        - avoid stockouts in July or Nov.



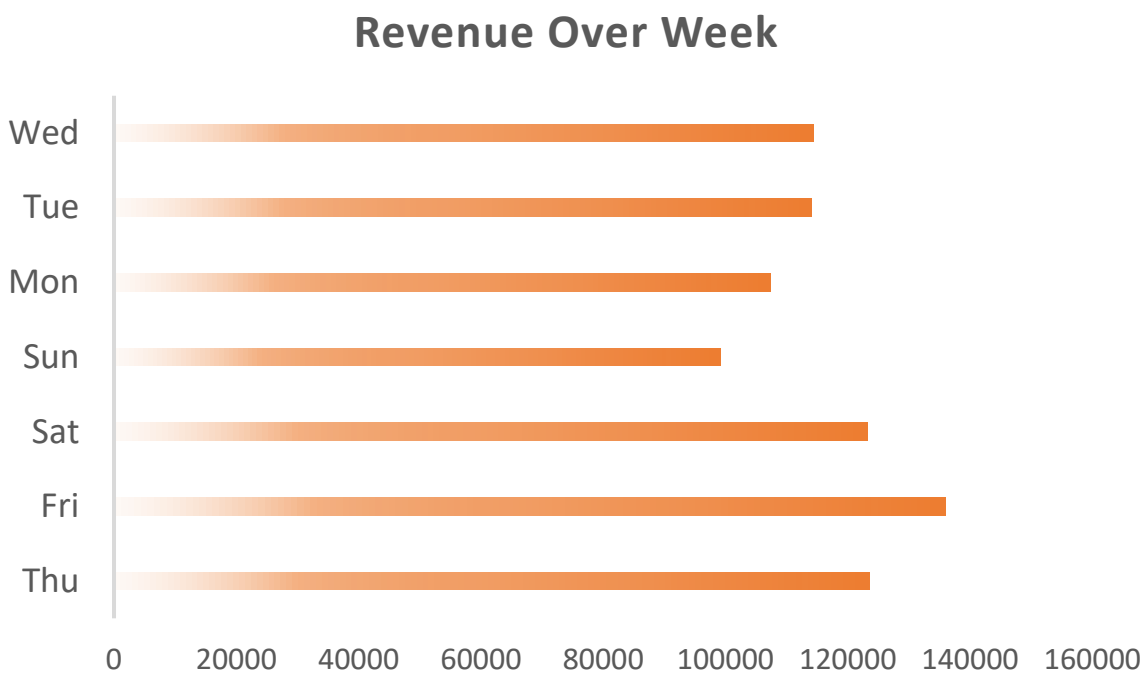
Key Insights:

- Classic pizza dominated with (~30%) of total orders making them the most preferred category.
- Supreme and veggie contributes equally (~24%) indicate stable mid-level demand.
- Chicken category underperforming.

Recommendation:

- High demand
  - Promote classic pizzas as flagship products in marketing campaigns.
  - Introduce premium variants or upper size combos to increase revenue from high demand
- Low demand
  - Reposition chicken pizza with new flavors, offers or combos
  - Offer 15% discount on chicken category.
  - Run targeted discount or bundle deals to improve sales share
- Stable demand
  - Maintain consistent quality & availability.
  - Veggie likely attracts a specific audience segment
  - Target veggie and supreme segmentation with personalized offers
  - Use ordering patterns to create customized recommendations.

- Focus on marketing of veggie and supreme category in order to improve overall profit margins

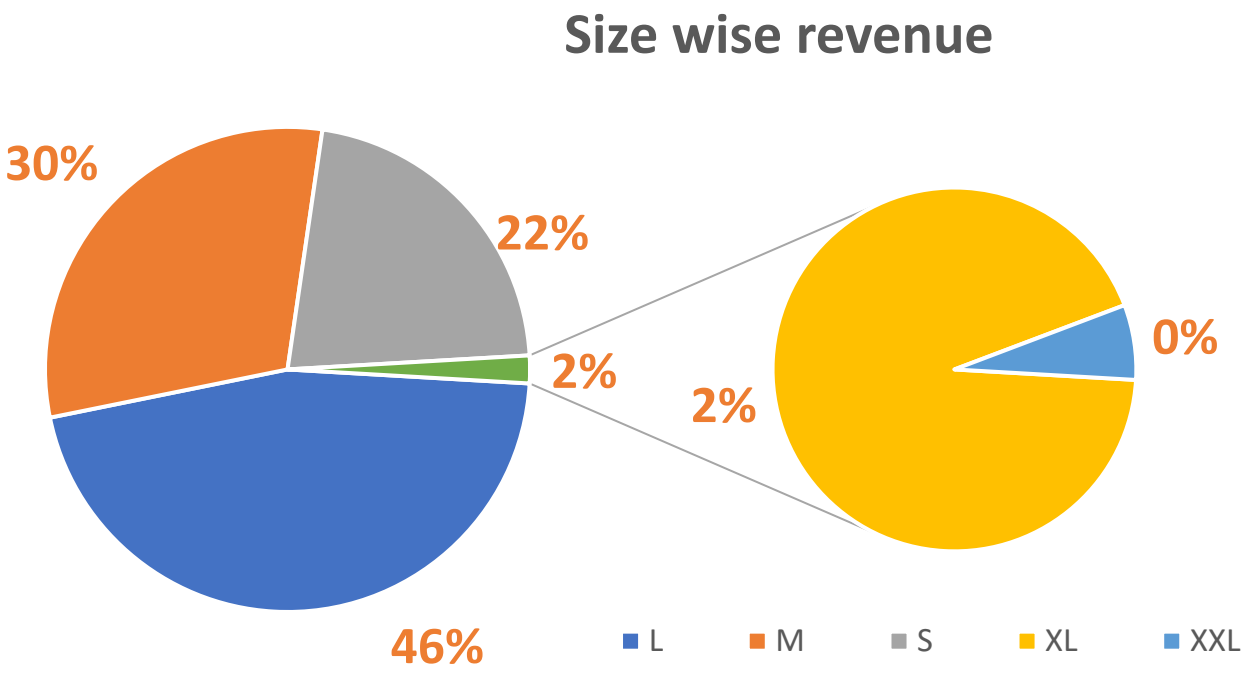


Key Insights:

- Highest demand - Friday, Saturday & thursday
- Lowest demand – Sunday & monday
- Monday-Wednesday are moderate but stable demand.

Recommendation:

- High performing days
  - focus on top selling items, remove and redesign for low performing items
  - avoid stockouts in Tuesday - Saturday
  - align staff shifts and inventory planning to perform smooth operation
  - premium combos and upscaling strategies to capitalize on high demand
- Low performing days
  - Launch & promotes ads before these days
  - Launch offers on these days
  - Launch Discount package on these days



- Key Insights
    - Large pizza dominates revenue (~46%) making them primary source of revenue
    - Medium contributes (~30%) secondary demand
    - Small contributes weak demand
    - XL and XXL contributes the least suggesting weakes demand

- Recommendation:
    - Promote large pizza as best value option
    - Large pizza dominates the most, shows demand for regular customer, bundle with drinks/beverages to increase average order value
    - Highlight (L) & (M) visually, most popular tag generates most of the revenue
    - Reprice and bundling small size as a personal meal deal
    - For XXL and XL target occasion, sport event or gathering

Problem Areas

- Off peak hours indicating low utilization of materials it affecting the proper utilization of staff(manpower) and resources fuels(electricity etc.)
- Chicken category shows weak performance compare to others
- Small Extra-large and Excess-extra-large contributing least indicates that number of orders and revenue share both are affecting
- During low demand days capacity isn't sufficiently using it leads of waste of materials

Business Recommendations:

- **Revenue optimization**
  - Focus on top-performing pizzas (high contribution group)
  - Introduce premium pricing for high-demand items
- **Menu engineering**
  - Redesign or remove bottom 20% low-performing pizzas
  - Reposition chicken category using combos or discounts
- **Demand optimization**
  - Lunch off-peak promotions (2-5 PM)
  - Introduce "Happy Hour" pricing to improve utilization
- **Operational efficiency**
  - Align staffing with peak hours
  - Reduce operational cost during low-demand hours
- **Upselling strategy**
  - Bundle large pizzas + beverages
  - Promote "Most popular" items visually in menu

Tools & skills used :

1. SQL: (MySQL) Workbench is 8.0.44 (released October 6, 2025)

   → Data Cleaning & Analysis

2. MS-Office 2021: Word, Excel

   → Dashboard & Visualization

3. Data Analysis: → Trend analysis, business insights, recommendations

Outcomes:

" This Project which is my very first complete project helped me to understand the data, how to clean analyses manipulate and explore the data and then turn that data into visuals in order to extract actionable insights and use the insights to give data-driven recommendations."

Author :-

Vishal

[LinkedIn] https://www.linkedin.com/in/vishal-mahavar-0a49043b3

[GitHub] https://github.com/vishal-tech3